



L6c: Iterative Methods for Solving Linear Algebraic Equations

Residual corrections, convergence, and matrix splittings
CHEME 5800 Cornell University

Objectives for Today

How can repeated updates solve a linear system, and when do the approximations approach the solution?

Today's concepts

- **Construct an iterative method**: derive a correction from a matrix splitting and distinguish satisfying a tolerance from reaching an iteration limit.
- **Assess convergence**: use the spectral radius, diagonal dominance, and an error bound to determine whether the iterates approach the solution.
- **Compare specific methods**: explain the updates and computational work of Jacobi, Gauss–Seidel, and relaxation.

Lecture: [Iterative Methods for Solving Linear Algebraic Equations](#)

Lecture and Example

Lecture: Iterative Methods for Solving Linear Algebraic Equations

Derive residual corrections, establish convergence conditions, and compare diagonal and triangular matrix splittings.

Example: Fun with Iterative Methods

Solve the same system with an iterative method and a direct solver. Check the computed solutions, then compare the solvers' runtimes and memory allocations.

Repeated corrections offer an alternative when a direct solution requires too much computation or memory.

General Iterative Method

Let $\mathbf{A} \in \mathbb{R}^{n \times n}$ be nonsingular, let $\mathbf{x} \in \mathbb{R}^n$ contain the unknowns, and let $\mathbf{b} \in \mathbb{R}^n$ be the right-hand side. We seek the unique solution of:

$$\mathbf{Ax} = \mathbf{b}.$$

An **iterate** $\mathbf{x}^{(k)}$ is the approximation after k updates, starting from the initial guess $\mathbf{x}^{(0)}$. The **residual** at this iterate is:

$$\mathbf{r}^{(k)} = \mathbf{b} - \mathbf{Ax}^{(k)}.$$

Each component of the residual measures the mismatch in one equation. If the residual is zero, the iterate is the exact solution.

The **solution error** is the difference between the approximation and the exact solution. A small residual alone does not guarantee a small solution error.

A Matrix Splitting Defines the Correction

Choose a nonsingular matrix $\mathbf{M}$ whose systems are inexpensive to solve. A **matrix splitting** writes $\mathbf{A}$ as:

$$\mathbf{A} = \mathbf{M} - \mathbf{N}, \quad \mathbf{N} = \mathbf{M} - \mathbf{A}.$$

A **correction** $\mathbf{d}^{(k)}$ is the vector added to the current approximation. We compute the correction and update the approximation using:

$$\underbrace{\mathbf{M}\mathbf{d}^{(k)} = \mathbf{r}^{(k)}}_{\text{solve for the correction}}, \quad \underbrace{\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \mathbf{d}^{(k)}}_{\text{apply the correction}}.$$

A correction need not reduce the residual norm or the solution-error norm.

A fixed splitting defines a **stationary iterative method**. Choosing a diagonal or triangular $\mathbf{M}$ simplifies the correction solve.

General Iterative Method: Algorithm

Initialize: Choose $\mathbf{x}^{(0)}$, an absolute tolerance $\epsilon > 0$ for the Euclidean residual norm, and a nonnegative integer update limit `maxiter`. Set $k \leftarrow 0$ and `converged` $\leftarrow$ `false`.

While not `converged` **do**:

1. Calculate $\mathbf{r}^{(k)} \leftarrow \mathbf{b} - \mathbf{A}\mathbf{x}^{(k)}$.
2. Check whether to stop:
 - If $\|\mathbf{r}^{(k)}\|_2 < \epsilon$, **then**: return $\mathbf{x}^{(k)}$ with `converged` = `true`.
 - Otherwise, if $k \geq \text{maxiter}$, **then**: return $\mathbf{x}^{(k)}$ with `converged` = `false` and a warning.
3. Solve $\mathbf{M}\mathbf{d}^{(k)} = \mathbf{r}^{(k)}$.
4. Update $\mathbf{x}^{(k+1)} \leftarrow \mathbf{x}^{(k)} + \mathbf{d}^{(k)}$.
5. Increment $k \leftarrow k + 1$.

The counter k records completed updates. The `converged` flag reports whether the residual tolerance was met. Both return statements terminate the algorithm.

Update Direction: The Stationary Form

Substitute the splitting into the system, then use the current approximation on the right-hand side:

$$(\mathbf{M} - \mathbf{N})\mathbf{x} = \mathbf{b},$$

$$\mathbf{M}\mathbf{x} = \mathbf{b} + \mathbf{N}\mathbf{x},$$

$$\mathbf{M}\mathbf{x}^{(k+1)} = \mathbf{b} + \mathbf{N}\mathbf{x}^{(k)}.$$

Define the iteration matrix $\mathbf{G}$ and constant vector $\mathbf{c}$:

$$\mathbf{x}^{(k+1)} = \underbrace{\mathbf{M}^{-1}\mathbf{N}}_{\mathbf{G}} \mathbf{x}^{(k)} + \underbrace{\mathbf{M}^{-1}\mathbf{b}}_{\mathbf{c}}.$$

The matrix $\mathbf{G}$ and vector $\mathbf{c}$ remain fixed as k increases. We can also express this update as a correction added to the current approximation.

Update Direction: A Residual Correction

With $\mathbf{I}$ the identity matrix, substitute $\mathbf{N} = \mathbf{M} - \mathbf{A}$ into the stationary update:

$$\begin{aligned}\mathbf{x}^{(k+1)} &= \mathbf{M}^{-1}\mathbf{b} + (\mathbf{I} - \mathbf{M}^{-1}\mathbf{A})\mathbf{x}^{(k)} \\ &= \mathbf{x}^{(k)} + \mathbf{M}^{-1} \underbrace{(\mathbf{b} - \mathbf{A}\mathbf{x}^{(k)})}_{\text{residual } \mathbf{r}^{(k)}} \\ &= \mathbf{x}^{(k)} + \underbrace{\mathbf{M}^{-1}\mathbf{r}^{(k)}}_{\text{correction } \mathbf{d}^{(k)}}.\end{aligned}$$

Thus, the correction solves $\mathbf{M}\mathbf{d}^{(k)} = \mathbf{r}^{(k)}$.

The exact solution is a **fixed point**: its residual is zero, so the update leaves the solution unchanged. We must still determine whether iterates starting from other initial guesses approach this fixed point.

Convergence: Propagation of the Error

Let $\mathbf{x}^*$ be the exact solution and define the solution error by $\mathbf{e}^{(k)} = \mathbf{x}^{(k)} - \mathbf{x}^*$. Subtract the fixed-point equation from the update:

$$\begin{aligned}\mathbf{e}^{(k+1)} &= (\mathbf{G}\mathbf{x}^{(k)} + \mathbf{c}) - (\mathbf{G}\mathbf{x}^* + \mathbf{c}) \\ &= \mathbf{G}\mathbf{e}^{(k)}, \\ \mathbf{e}^{(k)} &= \mathbf{G}^k \mathbf{e}^{(0)}.\end{aligned}$$

The residual and the solution error are related by $\mathbf{r}^{(k)} = -\mathbf{A}\mathbf{e}^{(k)}$.

Convergence means $\mathbf{e}^{(k)} \rightarrow \mathbf{0}$ as $k \rightarrow \infty$. The iteration converges from every initial guess if and only if $\mathbf{G}^k \rightarrow \mathbf{0}$.

The Spectral-Radius Convergence Condition

Let λ_i denote the eigenvalues of $\mathbf{G}$. The **spectral radius** of $\mathbf{G}$ is the largest eigenvalue magnitude:

$$\rho(\mathbf{G}) = \max_i |\lambda_i|.$$

The stationary iteration converges to the solution from every initial guess if and only if:

$$\rho(\mathbf{G}) < 1.$$

The condition $\rho(\mathbf{G}) < 1$ guarantees that the error approaches zero. The error norm may nevertheless increase during some updates.

Derivation: [Error propagation and the spectral-radius criterion](#)

Convergence Need Not Be Monotone

Take the following iteration matrix and initial error:

$$\mathbf{G} = \begin{bmatrix} 1/2 & 2 \\ 0 & 1/2 \end{bmatrix}, \quad \mathbf{e}^{(0)} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

Both eigenvalues of $\mathbf{G}$ are $1/2$, but the first update increases the error norm:

$$\mathbf{e}^{(1)} = \begin{bmatrix} 2 \\ 1/2 \end{bmatrix}, \quad \|\mathbf{e}^{(1)}\|_2 = \frac{\sqrt{17}}{2} \approx 2.06 > 1.$$

Repeated multiplication nevertheless gives:

$$\mathbf{G}^k = 2^{-k} \begin{bmatrix} 1 & 4k \\ 0 & 1 \end{bmatrix} \rightarrow \mathbf{0}.$$

The factor 2^{-k} decays faster than k grows, so the error approaches zero.

Diagonal Dominance

Write a_{ij} for entry (i, j) of $\mathbf{A}$. Strict diagonal dominance by rows requires:

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}|, \quad i = 1, \dots, n.$$

The magnitude of each diagonal entry exceeds the sum of the magnitudes of the other entries in that row.

- This condition guarantees convergence of Jacobi and Gauss–Seidel from every initial guess.
- The inequality must hold in every row.
- Strict diagonal dominance is sufficient for convergence, but it is not necessary.

Why Diagonal Dominance Works for Jacobi

Assume that $\mathbf{A}$ is strictly diagonally dominant by rows. Let $\mathbf{D}$ be the diagonal matrix formed from the diagonal entries of $\mathbf{A}$. Jacobi uses $\mathbf{M} = \mathbf{D}$ and $\mathbf{N} = \mathbf{D} - \mathbf{A}$, giving:

$$\mathbf{G}_J = \mathbf{D}^{-1}(\mathbf{D} - \mathbf{A}), \quad \|\mathbf{G}_J\|_\infty = \max_i \frac{\sum_{j \neq i} |a_{ij}|}{|a_{ii}|} < 1.$$

The induced infinity-norm is the largest absolute row sum. For any eigenpair $(\lambda, \mathbf{v})$ with $\mathbf{v} \neq \mathbf{0}$:

$$|\lambda| \|\mathbf{v}\|_\infty = \|\mathbf{G}_J \mathbf{v}\|_\infty \leq \|\mathbf{G}_J\|_\infty \|\mathbf{v}\|_\infty, \\ \rho(\mathbf{G}_J) \leq \|\mathbf{G}_J\|_\infty < 1.$$

Thus, strict row diagonal dominance guarantees convergence of Jacobi. Proving convergence of Gauss–Seidel requires a separate argument.

Rate of Convergence and an Error Bound

Suppose an induced matrix norm satisfies $q = \|\mathbf{G}\| < 1$. Use its corresponding vector norm to bound the error:

$$\begin{aligned}\|\mathbf{e}^{(k)}\| &= \|\mathbf{G}^k \mathbf{e}^{(0)}\| \\ &\leq \|\mathbf{G}^k\| \|\mathbf{e}^{(0)}\| \\ &\leq q^k \|\mathbf{e}^{(0)}\|.\end{aligned}$$

The **contraction factor** q satisfies $\|\mathbf{e}^{(k+1)}\| \leq q \|\mathbf{e}^{(k)}\|$. A smaller q gives a smaller upper bound for the same initial error.

If the chosen norm gives $q \geq 1$, the bound does not show that the error approaches zero, even when $\rho(\mathbf{G}) < 1$.

With $q < 1$, each update reduces the error norm until the error is zero.

A Sufficient Iteration Count

Let $E_0 \geq \|\mathbf{e}^{(0)}\|$, with $E_0 > 0$. Choose a solution-error tolerance $0 < \epsilon_e < E_0$ and assume $0 < q < 1$. Requiring $q^k E_0 \leq \epsilon_e$ gives:

$$k \ln q \leq \ln(\epsilon_e / E_0),$$

$$k \geq \left\lceil \frac{\ln(E_0 / \epsilon_e)}{-\ln q} \right\rceil.$$

Dividing by $\ln q < 0$ reverses the inequality. The ceiling rounds the count up to an integer. The error may reach the tolerance in fewer updates.

For $q = 0.5$, $E_0 = 1$, and $\epsilon_e = 10^{-3}$, the bound gives **10 updates**, since $2^{-10} \approx 9.77 \times 10^{-4}$.

Scope: ϵ_e bounds solution error; the algorithm uses residual tolerance ϵ . The initial error bound E_0 may be unknown.

Specific Methods

Split $\mathbf{A}$ into its diagonal matrix $\mathbf{D}$, strictly lower triangular part $\mathbf{L}$, and strictly upper triangular part $\mathbf{U}$:

$$\mathbf{A} = \mathbf{D} + \mathbf{L} + \mathbf{U}.$$

With nonzero diagonal entries and relaxation factor $\omega > 0$, the matrices used in the correction solves are:

$$\mathbf{M}_J = \mathbf{D}, \quad \text{Jacobi,}$$

$$\mathbf{M}_{GS} = \mathbf{D} + \mathbf{L}, \quad \text{Gauss-Seidel,}$$

$$\mathbf{M}_\omega = \frac{1}{\omega} \mathbf{D} + \mathbf{L}, \quad \text{successive over-relaxation.}$$

Each method converts the same residual into a different correction.

Jacobi: Independent Component Updates

For Jacobi, the correction solves $\mathbf{D}\mathbf{d}^{(k)} = \mathbf{r}^{(k)}$. Solving each row gives the corresponding component of the next iterate:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j^{(k)} \right), \quad i = 1, \dots, n.$$

Every component update uses the **previous iterate** $\mathbf{x}^{(k)}$. Keep $\mathbf{x}^{(k)}$ unchanged until all components of $\mathbf{x}^{(k+1)}$ are ready.

The iteration matrix is:

$$\mathbf{G}_J = -\mathbf{D}^{-1}(\mathbf{L} + \mathbf{U}).$$

Component updates can run in parallel. Strict row diagonal dominance guarantees convergence.

Algorithm: [The Jacobi Method](#)

Gauss–Seidel: Sequential Component Updates

Solve $(\mathbf{D} + \mathbf{L})\mathbf{d}^{(k)} = \mathbf{r}^{(k)}$ by forward substitution. The equivalent component update is:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j < i} a_{ij} x_j^{(k+1)} - \sum_{j > i} a_{ij} x_j^{(k)} \right).$$

When updating component i , use the **new values** for components $j < i$ and the values from the previous iterate for components $j > i$. The iteration matrix is:

$$\mathbf{G}_{GS} = -(\mathbf{D} + \mathbf{L})^{-1}\mathbf{U}.$$

Strict row diagonal dominance guarantees convergence. Reusing updated values can reduce the iteration count, but it introduces dependencies between updates.

Algorithm: [The Gauss–Seidel Method](#)

Successive Over-Relaxation

The relaxation factor ω changes each sequential component update. The correction solves:

$$(\mathbf{D} + \omega \mathbf{L}) \mathbf{d}^{(k)} = \omega \mathbf{r}^{(k)}.$$

Applying the correction gives:

$$x_i^{(k+1)} = (1 - \omega)x_i^{(k)} + \frac{\omega}{a_{ii}} \left(b_i - \sum_{j < i} a_{ij}x_j^{(k+1)} - \sum_{j > i} a_{ij}x_j^{(k)} \right).$$

When updating component i , the components $j < i$ already contain their relaxed values. The iteration matrix is:

$$\mathbf{G}_\omega = (\mathbf{D} + \omega \mathbf{L})^{-1}((1 - \omega)\mathbf{D} - \omega \mathbf{U}).$$

Algorithm: [Successive Over-Relaxation](#)

Relaxation and Convergence

The relaxation factor ω scales the change from the previous value of a component to the value obtained by solving its row equation:

- $0 < \omega < 1$: **under-relaxation**, taking a partial step.
- $\omega = 1$: the Gauss–Seidel update.
- $1 < \omega < 2$: **over-relaxation**, stepping beyond the row-equation solution.

If $\mathbf{A}$ is **symmetric positive definite**, convergence is guaranteed for $0 < \omega < 2$. Strict diagonal dominance alone does not guarantee convergence over that entire interval.

A suitable factor can reduce the iteration count. The best choice depends on the matrix; formulas for an optimal factor need additional assumptions.

Reading: [Choosing the relaxation factor; special-case formula in the algorithm notebook.](#)

Work per Iteration

A **sweep** updates all n components once. For these methods, each completed iteration checks the residual and performs one full sweep.

Method	Correction work and dependencies
Jacobi	Diagonal solve; components use the same previous iterate.
Gauss–Seidel	Lower triangular solve; later rows depend on earlier updates.
Relaxation	Relaxed lower triangular solve; the factor changes convergence.

For dense matrices, a full iteration requires work proportional to n^2 . Sparse implementations can work with the stored nonzero entries.

Runtime depends on both the **cost per iteration** and the **number of iterations** needed to meet the residual tolerance.

Example: Fun with Iterative Methods

The example constructs a symmetric, strictly diagonally dominant matrix with positive diagonal entries. These properties make the matrix positive definite.

1. Check symmetry and positive diagonal entries. Verify strict diagonal dominance in **every row**.
2. Solve the system with an iterative method and a direct solver. Check the final residual norm and the norm of the difference between solutions.
3. Benchmark both solvers with the same settings used above. Compare the timing distributions and memory allocations.

The iterative solver stores every iterate, adding to its runtime and memory use. The comparison applies to this implementation and test system.

Example: [Fun with Iterative Methods](#)

Lab: Oxygen Delivery in a Tissue Slice

In [L6d](#), solve the dimensionless steady oxygen balance in a tissue slice with Jacobi, Gauss–Seidel, and successive over-relaxation.

- Compare each result with the **direct solution**.
- Use **residual-norm histories** to check whether the residual tolerance was met.
- Compare iteration counts, then change the **Thiele modulus** and the edge oxygen supply.

Interpret speed only after checking the solution. A smaller iteration count is meaningful when the methods meet the same residual tolerance for the same system and initial guess.

Lab: [Oxygen delivery in a tissue slice](#)

Summary

We developed stationary iterative methods by converting the residual into a correction through a matrix splitting.

Key takeaways

- **Residual-based corrections.** The splitting determines the correction solve. Reaching an iteration limit does not establish convergence.
- **Convergence and error bounds.** The iteration matrix characterizes convergence. Diagonal dominance and contracting norms provide useful sufficient conditions and bounds.
- **Choosing a method.** Independent, sequential, and relaxed updates affect both computational work and convergence.

The example and lab connect these results to measured residual norms, iteration counts, and runtimes.